

Chapter 1 — Review Questions

100 multiple choice questions, grouped by topic

1 Review Questions

Difficulty is tagged [E] easy, [M] medium, [H] hard. These questions test *understanding*: in most of them, at least one wrong option is what you would believe if you *almost* understood — read carefully.

1.1 Abstraction (1–8)

1. [E] An abstraction barrier lets you: a) skip ever having to learn anything about what’s below it, since it no longer matters b) use something through its inputs and outputs without re-deriving its internals c) prove a program correct by checking each layer beneath it one at a time d) make code run faster by hiding the slow parts from the compiler
2. [M] You use a library’s `sort()` without knowing its algorithm. The abstraction “leaks” when: a) sorting a million items is unexpectedly slow and you must ask *why* b) you read its documentation to learn which algorithm the authors chose c) you call it twice in one program and get the same ordering both times d) the library gets updated and its internals change without your code noticing
3. [M] The wallpaper/group theory story is in this chapter because: a) there are exactly 17 useful algorithms, one for each wallpaper group b) crystallography is a major source of the data sets data scientists analyze c) it shows the same abstract rules governing physically unrelated things d) the mathematicians who classified the wallpaper patterns went on to invent computing
4. [E] In the chapter’s terms, a data scientist at a biology company is: a) required to learn the biology first, since you can’t analyze what you don’t understand b) outside all abstraction barriers, since data science applies to every field c) below the biologists’ abstraction barrier, supplying the layer they build on d) an abstraction barrier to the biologists, and they to her
5. [H] “Working comfortably above and below abstraction” means knowing when to: a) dig all the way to the bottom before trusting any tool with real data b) treat a tool as a black box, and when to open the box because the box is the problem c) trust the documentation over your own experiments, and when to do the reverse d) rewrite a library yourself rather than depend on someone else’s code
6. [E] The chapter builds small versions of standard software because: a) understanding what’s underneath makes you better at using the real thing b) professionals are routinely expected to write these components from scratch c) libraries change too fast for it to be worth learning any one of them d) the standard versions hide too many bugs to be trusted in scientific work

7. [M] Which is NOT an example of abstraction from the chapter? a) a gate symbol standing for a transistor circuit b) a file standing for scattered disk blocks c) an IP address standing for a route through routers d) a faster clock speed standing for more transistors
8. [M] The driving-a-car analogy claims that knowing what an engine does: a) is equivalent to knowing the thermodynamics the engine is built on b) is required before you can legally or safely drive at all c) makes you a better driver without needing to build engines d) only matters for mechanics, since drivers never touch the engine

1.2 Charge, Transistors, and Gates (9–20)

9. [E] In the water analogy, voltage corresponds to: a) the total amount of water stored in the system's pipes b) the pipe's diameter, which sets how much can flow at once c) the pressure pushing water through the pipe d) the water itself, flowing from high places to low places
10. [E] A transistor is best described as: a) an amplifier that makes weak electrical signals strong enough to use b) a switch controlled by an electrical input c) a storage element that holds one bit as trapped charge d) a source of electrons that feeds current into the circuit
11. [M] Why does a computer need a clock at the level of wires and switches? a) to define *when* the value on a wire counts as a 0 or 1 b) to slow the electrons down enough that the transistors can react to them c) to keep all the gates supplied with a steady stream of power d) to keep the time of day for programs that need dates and timestamps
12. [E] An AND gate outputs 1 when: a) exactly one input is 1 b) both inputs are 0 c) either input is 1, or both d) both inputs are 1
13. [M] Two switches in *series* between voltage and output implement: a) OR b) AND c) XOR d) NOT
14. [M] Two switches in *parallel* implement: a) AND b) NAND c) OR d) XOR
15. [E] XOR outputs 1 exactly when: a) the inputs differ b) either input is 1, including both c) both inputs are 0 d) both inputs are 1
16. [M] NAND outputs 0 only when both inputs are 1. This makes NAND: a) an OR whose inputs have each been inverted first b) useless for arithmetic, which is why adders avoid it c) the same as NOT, but applied to two inputs at once d) an AND followed by an inversion
17. [H] “After introducing gate symbols we never draw transistors again.” The deeper point is: a) transistors stopped mattering to how the machine actually computes b) a solved layer can be *sealed*: correctness above no longer depends on re-checking below c) gate symbols are electrically accurate diagrams of the transistor wiring d) real circuit designs are proprietary, so textbooks must use the symbols instead
18. [M] The chapter says you are NOT expected to design gate circuits. You ARE expected to: a) know that gates reduce to a handful of switches — no magic b) be able to follow a chip's fabrication process from sand to packaged part c) derive the axioms of Boolean algebra from the behavior of switches d) memorize the standard transistor configurations for each of the basic gates
19. [H] A student claims: “a 1 in a wire means electrons are stored in the wire.” What's wrong? a) electrons are stored in the transistors, and the wires merely connect them b) nothing — that's

a fair description of how digital circuits hold their state c) a 1 is a *flow* (or voltage level) read at a clock tick, not a stored quantity d) it's backwards — a 1 means the wire has been emptied of its electrons

20. [E] The first abstraction barrier in the chapter's ladder seals away: a) the operating system and its services b) binary numbers and their arithmetic c) the logic gates and their truth tables d) electron flow and semiconductor physics

1.3 Binary and Representation (21–32)

21. [E] The number of distinct patterns in 8 bits is: a) 8 b) 128 c) 255 d) 256

22. [M] The largest *unsigned* value 8 bits can hold is: a) 256, since 8 bits give 256 values b) 255 c) 128, the value of the highest bit d) 127, once the sign bit is set aside

23. [M] Binary 0101 1011 equals: a) 91 b) 93 c) 87 d) 101

24. [E] Counting in binary, the number after 0111 is: a) 0110 b) 0112 c) 1000 d) 1111

25. [M] Binary carrying works “just like decimal” means: a) you carry whenever a digit would exceed 9, exactly as you learned in school b) you carry when a digit would exceed its maximum symbol — here, 1 c) binary never needs carrying, because a digit can only be 0 or 1 d) you carry on every second digit, since binary digits come in pairs

26. [H] A signed 8-bit integer ranges -128 to $+127$, not -128 to $+128$, because: a) zero occupies one of the 256 patterns b) the sign bit stores a -1 that must be subtracted from the positive range c) the pattern for 128 is reserved by the hardware for signaling errors d) positive numbers need an extra bit of space that negative numbers don't

27. [M] “The same data can be represented multiple ways.” Which pair illustrates this? a) a file on disk vs. the name that appears in the directory b) a value held in RAM vs. the copy of it held in cache c) a number stored in binary vs. its decimal printout d) two different numbers sharing one register at different times

28. [E] To a computer, an image is: a) a file extension that tells programs to open a viewer b) a rectangle of light whose colors exist only on the screen c) a picture it recognizes by matching it against ones it has seen d) numbers encoding pixel colors

29. [H] Language models “represent knowledge in vector spaces” connects to this chapter because: a) vectors are stored in RAM, and this chapter explains how RAM works b) it's another choice of representation, where similarity becomes geometric distance c) neural networks compute in binary, and vectors are how binary encodes words d) each word gets one 8-bit pattern, exactly like the characters in a text file

30. [M] Why does positional notation (each digit worth $\text{base}^{\text{position}}$) matter? a) the same simple rule scales to numbers of any size, in any base b) it guarantees the fewest digits any number system could ever use c) computers are physically unable to store numbers any other way d) it's the only number system in which arithmetic is possible at all

31. [E] A byte is: a) the amount of charge one transistor can store b) any binary number, whatever its length c) 8 bits d) the smallest block a disk can read or write

32. [H] A student says “binary numbers are less precise than decimal.” The right response: a) true, but only once the numbers grow past what 64 bits can hold b) false — binary is *more* precise, which is why computers use it c) true — that imprecision is exactly why floating point had to be invented d) false — precision is about *how many digits you keep*, not the base

1.4 Arithmetic and the Adder (33–40)

33. [E] In a half adder, the sum bit comes from: a) AND b) OR c) XOR d) NAND

34. [E] The carry bit comes from: a) XOR b) AND c) OR d) NOT

35. [M] Why XOR for the sum? Because addition of two bits: a) is 1 exactly when the bits differ b) is 1 when both bits are 1, and 0 in every other case c) requires subtracting the carry back out of the total d) can never produce a 1 in the same column as the carry

36. [M] $1 + 1$ in a half adder gives sum 0, carry 1 — which in binary reads: a) $11 = 3$ b) $00 = 0$ c) $01 = 1$ d) $10 = 2$

37. [E] To add numbers wider than one bit, you: a) use physically bigger gates that accept the wider numbers whole b) chain adders, feeding each carry into the next column c) convert both numbers to decimal, add, and convert the result back d) route each column through a multiplexer to a shared adder

38. [H] The deep claim of this section is that arithmetic is: a) a *pattern of logic* — gates arranged so their truth table IS the addition table b) performed by the operating system on behalf of the hardware c) approximate at the hardware level, with software correcting the errors d) built into silicon directly by the physical laws electrons obey

39. [M] A full adder differs from a half adder because it must also: a) store its result until the next tick of the clock b) subtract, so that the same circuit can handle negative numbers c) accept a carry *in* from the previous column d) multiply by two whenever its own carry fires

40. [H] If XOR gates were unavailable, a half adder’s sum could be built from AND/OR/NOT because: a) the sum doesn’t actually require XOR, only the carry does b) OR alone gives the right sum in every case except one that never occurs c) it couldn’t — XOR is a primitive that the other gates cannot reproduce d) $XOR = (A \text{ OR } B) \text{ AND NOT } (A \text{ AND } B)$

1.5 Memory and the von Neumann Machine (41–52)

41. [E] A flip-flop: a) performs one-bit addition, including the carry b) remembers a value until told otherwise c) routes a signal down one of two selectable paths d) generates the clock pulses that step the machine

42. [E] A multiplexer: a) routes a signal to a chosen destination b) inverts whichever input is currently selected c) adds its inputs and signals a carry when they overflow d) stores several bits and releases them one at a time

43. [M] $\text{RAM} = \text{multiplexers} + \text{flip-flops}$ means memory is: a) necessarily slower than the logic, since remembering takes an extra step b) a separate substance, distinct from the switching logic around it c) routing plus remembering — the same components as everything else d) the one part of the computer that isn’t ultimately made of switches

44. [M] “Random access” in RAM means: a) the OS scatters programs across addresses chosen at random for safety b) reads are only *statistically* reliable, so important data is read twice c) the contents come up randomized at every boot until initialized d) any address can be reached directly, without reading in order
45. [E] A register is: a) a small, fast holding place for values being worked on b) the list the OS keeps of every installed program c) the region of the disk where the OS records its settings d) a slower, larger kind of cache shared by all the cores
46. [M] What turns the calculator into a computer? a) more registers, so it can hold enough intermediate results to be useful b) instructions stored in memory + a clock stepping through them automatically c) a faster adder, fast enough to finish before the next key press d) attaching a keyboard so a person can feed it operations continuously
47. [E] In the von Neumann architecture, memory holds: a) data only b) instructions only c) both instructions and data d) the operating system only
48. [M] The program counter: a) counts clock ticks so the OS can measure processor speed b) tracks how much of its time allotment a program has consumed c) counts how many programs the machine has run since boot d) holds the address of the current instruction
49. [H] Control flow exists because some instructions: a) run faster than others, so the order must adapt as the program runs b) write a new value into the program counter, possibly conditionally c) are stored outside memory, where the normal sequence can’t reach them d) bypass the ALU entirely and act on the memory unit directly
50. [H] A student says: “the computer knows which memory holds instructions and which holds data.” The von Neumann answer: a) false — *being pointed at by the program counter* is what makes bits an instruction b) true — every memory word carries a marker bit saying which kind it is c) false — instructions don’t live in memory at all; they live inside the ALU d) true — memory is physically divided into an instruction half and a data half
51. [M] An instruction like “ADD addr8, addr9 → addr12” contains: a) the result of the previous instruction, passed along to this one b) the actual numbers to add, copied into the instruction when it was written c) *addresses* of the operands and destination, plus the operation d) the operation only — the machine finds the operands wherever they happen to be
52. [E] The chapter’s point in building the computer step by step is: a) computers used to be simple enough to understand, and no longer are b) each layer of hardware was a historical accident that could have gone otherwise c) you should build one yourself to really understand the machine d) nothing mystical is added at any layer — it’s switches all the way down

1.6 Chips, Clocks, and FLOPS (53–62)

53. [E] Lithography builds chips by: a) carving the circuit into silicon with microscopic diamond drills b) projecting circuit patterns with light and applying chemistry in layers c) weaving microscopic wires into a lattice, layer over layer d) growing the transistors chemically from a seed pattern in solution
54. [M] Clock speeds stopped climbing around 4 GHz mainly because of: a) heat — faster switching dissipates more power than the chip can shed b) the speed of light, which a signal crossing the chip

cannot beat c) demand — beyond 4 GHz there was nothing left that needed the speed d) patents that locked the key techniques up with a few manufacturers

55. [M] Smaller transistors help because they: a) switch more reliably, allowing error checking to be removed b) are cheaper to design, since the layouts can simply be scaled down c) need less power, so more can run in the same heat budget d) hold more data each, multiplying the chip's storage as they shrink

56. [E] The industry's response to the clock-speed wall was: a) returning to simpler 16-bit designs that generate less heat b) moving arithmetic off the CPU and into the memory chips c) making liquid cooling standard equipment on consumer machines d) multiple cores per chip

57. [M] The same M-series chip performs differently in an iPad vs. a Mac Pro because: a) the Mac version is manufactured on a newer process with faster silicon b) the enclosure determines how much power and cooling it gets c) the iPad's license deliberately throttles it to protect Mac sales d) it doesn't differ — the benchmarks measure the software, not the chip

58. [M] Caches exist because: a) going off-chip for data costs time and energy, so keep local copies b) disks are too small to hold everything programs need at once c) programs request data so randomly that prediction is the only option d) RAM occasionally corrupts data, and the cache holds verified copies

59. [H] Fetching memory in blocks pays off when: a) the cache has been disabled and every access must go to RAM anyway b) access is perfectly random, since blocks then sample memory evenly c) you're about to use neighboring data — the rest of the block is already there d) memory is mostly empty, so whole blocks can be moved without conflict

60. [M] Apple's unified memory avoids a specific cost, namely: a) maintaining the caches, which separate memories would each require b) the memory controller, which separate CPU and GPU memories each need c) paying RAM vendors for chips that Apple can fabricate itself d) copying data back and forth between CPU memory and GPU memory

61. [E] FLOPS measures: a) failed operations caught per second b) floating point operations per second c) file loads and stores per second d) the count of transistors doing arithmetic

62. [H] $\text{FLOPS} = \text{cores} \times \text{clock} \times \text{operations-per-cycle}$. Doubling cores rather than clock is attractive because: a) it raises FLOPS without raising per-transistor switching rates and heat b) the clock is fixed by the memory and cannot be changed independently c) every program automatically runs twice as fast on twice the cores d) extra cores are nearly free, since the silicon is already paid for

1.7 Floating Point (63–68)

63. [E] Floating point is essentially: a) a dedicated region of memory reserved for fractional values b) arithmetic with unlimited digits, at the cost of speed c) scientific notation in binary d) integer arithmetic with the decimal point tracked separately in software

64. [M] A 32-bit float allocates its bits as: a) 8 sign / 8 exponent / 16 fraction b) 32 fraction, with the exponent stored in a second word c) 16 fraction / 16 exponent d) 1 sign / 8 exponent / 23 fraction

- 65. [H]** A student says floats can represent any decimal exactly. Reality: a) true for any value up to 2^{32} , the format's natural limit b) false — only finite binary fractions fit; 0.1 is already inexact c) true as long as the value is positive and the exponent is in range d) false, but only for irrational numbers like π that no format can hold
- 66. [M]** The exponent field buys floating point its: a) enormous *range* b) speed c) sign d) precision
- 67. [M]** Quantizing neural networks to fewer bits (and it still works) suggests: a) the missing bits are recovered by the hardware during inference b) the networks were broken in ways the extra precision had been hiding c) the task doesn't need all the precision the training format carried d) floats and integers are interchangeable once a network is trained
- 68. [E]** Most scientific computing uses: a) decimal digits stored as text b) exact fractions of two integers c) 8-bit integers, for speed d) 32- or 64-bit floats

1.8 Storage and File Systems (69–78)

- 69. [E]** RAM vs. storage: the fundamental difference is: a) speed — they are the same technology at different price points b) RAM forgets when power is lost; storage persists c) price — storage is simply RAM bought in cheaper bulk d) age — RAM is the newer technology and will eventually replace storage
- 70. [M]** A disk sector is: a) the minimum unit read or written, with a physical address b) a damaged region the file system has fenced off from use c) the physical location where a single bit is magnetized d) one of the four quarter-circle regions the platter is divided into
- 71. [M]** “Formatting a disk” essentially means: a) installing the small piece of the OS that disks need in order to work b) writing zeros over every bit so no trace of the old data remains c) laying down the addressing grid that organizes the sectors d) certifying the platter surface by scanning it for physical defects
- 72. [E]** SSDs beat spinning disks on speed because: a) they sit closer to the CPU on a faster electrical connection b) they store data in a compressed form that takes less time to read c) their platters spin many times faster than a mechanical disk's can d) nothing mechanical has to move
- 73. [H]** Data centers still use spinning disks and tape because: a) they fail far less often, and in a data center reliability wins b) per byte stored, they remain far cheaper — speed isn't the only axis c) SSDs lose their contents over time unless they're regularly powered d) the industry's software was built around them and migration is risky
- 74. [M]** Swap is: a) the OS moving memory blocks to disk when RAM runs out, transparently b) a small cache on the CPU that trades entries with the one below it c) the reserve of spare sectors a disk swaps in when others fail d) the OS exchanging the contents of two files atomically
- 75. [H]** Your analysis suddenly runs 100× slower as data grows. This chapter's first suspect: a) the network, since data of that size is usually being streamed b) thermal throttling — the CPU has been running hot for too long c) you've exceeded RAM and the system is swapping to disk d) Python's memory manager, which slows down as its heap fragments
- 76. [M]** A partition is: a) a second disk the OS creates by mirroring the first b) the boundary the OS draws between its files and the user's c) the wall of reserved sectors separating two files on disk d) a region of one physical disk that can carry its own file system

77. [M] The chapter insists “the file is an invention” to make the point that: a) files are an aging idea that modern systems are already replacing b) even the most obvious-feeling abstractions were designed, and could be otherwise c) disks originally stored files natively, and modern systems emulate that d) whoever invents an abstraction controls the systems built on top of it

78. [M] The file table maps: a) file names to the disk blocks (and metadata) behind them b) each user to the files their permissions let them open c) running programs to the regions of memory they own d) logical sectors to the physical partitions that contain them

1.9 Boot and Operating Systems (79–88)

79. [E] The correct boot order is: a) OS → firmware → programs b) kernel → firmware → bootloader c) firmware → bootloader → kernel d) bootloader → BIOS → OS

80. [M] The bootloader must be tiny and special because: a) firmware verifies it byte by byte, and verification of large programs is slow b) it must fit entirely inside the CPU’s registers while it runs c) it shares its disk sector with the partition table, which limits its size d) it runs before any file system exists — it can’t “open a file” to find the kernel

81. [E] The operating system’s core job is: a) running the applications the user sees, starting with the browser b) managing hardware and sharing resources among programs c) translating programs into the machine instructions the CPU runs d) keeping files safe on disk and backed up against failure

82. [M] An API is: a) the set of functions programs call to ask the OS for services b) the program that installs software and registers it with the OS c) the protocol computers use to request services over the network d) the physical connector standard that chips use to talk to each other

83. [M] Programs are built “for” an OS because: a) each OS requires its own file extension, and loaders check for it b) each OS vendor requires programs to be certified before they will run c) a program is largely calls into one system’s specific API d) the hardware checks a program’s origin and refuses foreign ones

84. [E] A process is: a) one CPU core’s share of the work the machine is doing b) an open window and the program state behind it c) any file on disk that the OS knows how to execute d) a running program with its own allotment of memory and resources

85. [M] One core appears to run many programs at once because the OS: a) compresses each program so several fit into the core together b) switches between processes many times per second c) moves the extra programs to the GPU, which runs them in parallel d) runs each program only in the moments the others spend waiting

86. [H] *Preemptive* multitasking specifically adds: a) the OS can take the CPU back — a hung program can’t freeze the machine b) switching between processes fast enough to appear simultaneous c) support for several users running programs on one machine at once d) the ability to run more processes than the machine has memory for

87. [H] Why did Unix have users, permissions, and multitasking decades before home computers? a) the C language’s process model made those features nearly automatic b) its designers were academics, free to add features without market pressure c) mainframes were shared by many people

at once — those features weren't optional d) government contracts required security features in any funded system

88. [M] Swap, scheduling, and the API are all examples of the OS: a) spending resources on bookkeeping that slows every program down b) doing for free what mainframe systems once charged for by the hour c) taking over duties that programs used to implement for themselves d) being an abstraction barrier between programs and the machine

1.10 Unix and the Family Tree (89–94)

89. [M] “Unix is a standard, not just an OS” means: a) all Unixes, including Linux and macOS, share one original codebase b) Linux, BSD, macOS implement a common specification — there's even a certification c) Unix set the standard other systems copied, though it's no longer used itself d) the standard specifies the hardware a Unix kernel is entitled to expect

90. [M] A Linux distribution is: a) kernel + tools + packaging assembled into an installable whole (Ubuntu, Debian...) b) one release of the kernel, snapshotted and given a version number c) the network of mirrors that distributes Linux software worldwide d) the kernel alone, as distributed from the official repository

91. [M] macOS's kernel ancestry runs through: a) Android b) Windows NT c) NeXT and BSD d) DOS

92. [E] “Everything is a file” is powerful because: a) it lets the file system deduplicate identical data wherever it appears b) anything on the system can be backed up the way ordinary files are c) files are the fastest interface the operating system offers d) programs, devices, and networks all speak one interface — anything pipes into anything

93. [E] `ls`, `rm`, and `mkdir` are: a) built-in commands the shell recognizes and executes itself b) separate tiny programs the shell finds and runs c) kernel functions exposed directly to the user at the prompt d) abbreviations the shell expands into longer system commands

94. [H] Windows descends from CP/M-then-DOS rather than Unix. The chapter's structural consequence: a) multi-user, permissions, and the rest were retrofitted onto a single-user design b) Windows has no kernel in the Unix sense, only a collection of drivers c) Windows programs must carry their own copies of OS services with them d) Windows still cannot truly multitask the way Unix systems do

1.11 Networks (95–100 part 1)

95. [E] In the network stack, MAC addresses live at the _____ layer and IP addresses at the _____ layer: a) transport / application b) internet / link c) link / internet d) physical / transport

96. [M] Ports exist because: a) IP addresses are too short to identify every machine, so ports extend them b) routers need them to decide which packets are worth forwarding first c) machines routinely have several network cards that must be told apart d) one address may host many programs — the port picks which one gets the packet

97. [M] DNS translates: a) IP addresses to MAC addresses b) names to IP addresses c) packets to frames d) ports to programs

98. [H] A denial-of-service attack is dangerous *specifically because*: a) every individual request is legitimate — the system drowns in volume, not intrusion b) it steals data while the target is too busy to log the theft c) it rewrites the target’s DNS records while the servers are overloaded d) it breaks the encryption protecting every connection to the target

1.12 Mixing Systems (99–100)

99. [H] Rosetta 2 translates instructions; WSL 1 translated API calls. The difference in what they translate exists because: a) Windows programs make API calls while Mac programs are compiled to instructions b) Apple controls its hardware, so it could translate at a lower level than Microsoft c) they solve different mismatches — same OS/different chip vs. different OS/same chip d) Rosetta is a generation newer, and instruction translation had matured by then

100. [H] Opening a Colab notebook starts a container rather than a full virtual machine because: a) containers isolate users from each other more strongly than VMs can b) Python is interpreted, and interpreted languages cannot boot a full VM c) a virtual machine per user would require Google to license each OS copy d) a container shares the host kernel and packages only the app’s needs — near-instant start